DAY 1 Project Setup

Project: Predicting Physical and Electronic Properties of Crystalline Solids from their Text Descriptions

Category: CRYSTAL | Date: Monday, 19 Oct 2026

Plan: build it alone, in 20 days, push to GitHub every day.

GOAL OF TODAY

Get every tool ready + put the project on GitHub,
so from Day 2 we can start real work (data + code).

What we finished today:

- ✓ Created project folder LLM_PROP on Desktop
- ✓ Checked Python (3.11.9) and Git are installed
- ✓ Made the GitHub repo and connected our folder to it
- ✓ Made README.md + first commit + first push
- ✓ Made a virtual environment (venv)
- ✓ Installed pandas, numpy, matplotlib, scikit-learn, tqdm
- ✓ Installed PyTorch (CPU) 2.14.1 and transformers 5.20.1
- ✓ Added .gitignore + requirements.txt + notes.md and pushed

NOTE: This PDF explains every step I did today, in simple words, with the exact commands. Read it once before Day 2.

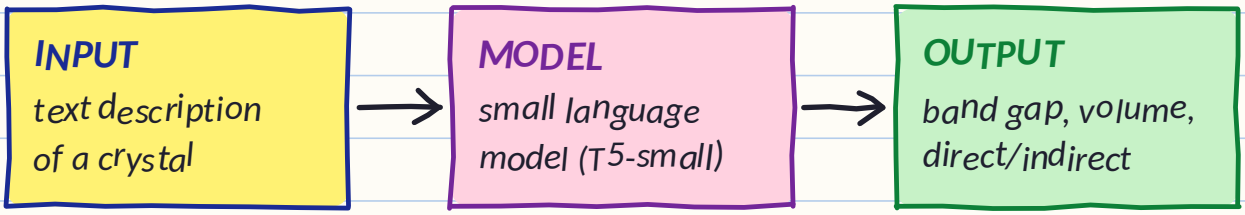
1. What is our project?

(in very simple words)

- A CRYSTAL is a solid whose atoms sit in a neat repeating pattern. Examples: salt, silicon, diamond.
- Scientists want its PROPERTIES, for example:
  - band gap - tells if it can conduct electricity
  - direct or indirect gap - how it behaves with light
  - unit cell volume - the size of the repeating box
- Testing in a lab is slow and costly, so we want AI to PREDICT them.

x OLD way: turn the crystal into a GRAPH (atoms = dots, bonds = lines) and use a GNN (graph neural network). Works, but is complicated and misses some info.

✓ NEW way (the LLM-Prop paper): describe the crystal in plain ENGLISH TEXT and let a small language model read it, like a human would.



What the paper claims (from the abstract):

- Paper = LLM-Prop, arXiv 2310.14029 (Rubungo, Arnold, Rand, Dieng, Oct 2023).
- It beats the best GNN by about 4% on band gap, 3% on direct/indirect, and 66% on volume.
- It also beats MatBERT (a science-trained BERT) with 3 times fewer parameters.
- Their dataset is called TextEdge (crystal text + properties).

REMEMBER: Our project = take the text of a crystal as INPUT, give its properties as OUTPUT. Then test if a SMALL model + smart text design still works well.

2. Setup steps (part 1)

folder, terminal, Python, Git

1 Make the project folder

On Desktop I created a folder named LLM_PROP. This is the home of the whole project.

Full path: C:\Users\<my name>\OneDrive\Desktop\LLM_PROP

2 Open a terminal INSIDE the folder

Open LLM_PROP in File Explorer > click the address bar at the top > type cmd > press Enter.

A black window opens and the path ends with LLM_PROP. That means I am in the right place.

3 Check Python

```
> python --version
```

output: `Python 3.11.9`

✓ 3.11.9 works perfectly for this project.

4 Check Git

```
> git --version
```

output: `git version 2.x.x  (Git was already installed)`

- Git = the tool that saves versions of my code and sends it to GitHub.

TIP: If 'python' or 'git' says 'not recognized', that tool is not installed. Install it first, then reopen the terminal.

3. Setup steps (part 2)

GitHub repo + connect my folder

5 Create the repository on GitHub.com

- Log in > click the + at top right > New repository.
- Name: LLM_PROP-66A7- (made under my organization LLM-prop)
- Description: Predicting crystal properties from text descriptions using small language models
- Choose Public.

x DO NOT tick 'Add a README' or anything else. Keep the repo empty.

- Click Create repository. Copy the link it shows.

My link: https://github.com/LLM-prop/LLM_PROP-66A7-.git

NOTE: It is an organization repo but I work alone. That is fine. The name ends with a dash, so I copy the link exactly as it is.

6 Connect my folder to GitHub

```
> git init
```

turns my folder into a Git project (creates a hidden .git folder)

```
> git remote add origin https://github.com/LLM-prop/LLM_PROP-66A7-.git
```

'remote' = link to GitHub. 'origin' = the nickname of that link

```
> git remote -v
```

shows the link twice: (fetch) and (push). Seeing both = connected!

4. My first commit

README -> add -> commit -> push

7 Create README.md

README.md = front page of the project on GitHub (title, problem, tech stack, paper link). I saved it in LLM_PROP, then checked:

```
> git status
```

output: `Untracked files: README.md (shown in red)`

- Red = Git sees the file but is not saving it yet.

8 Save it with a commit

```
> git add README.md
> git commit -m "Day 1: add README"
```

- add = put file in the 'to save' basket. commit = save a snapshot + message.

TIP: If Git says 'Please tell me who you are', run these once, then commit again: git config --global user.name "My Name" and git config --global user.email "me@example.com"

9 Send it to GitHub

```
> git branch -M main
> git push -u origin main
```

✓ A browser/login window may pop up. Sign in to GitHub. Then refresh the repo page: README.md is visible!



This 3-step loop = my daily GitHub update for 20 days!

5. Virtual environment + libraries

a clean box just for this project

10 Make and activate the venv

```
> python -m venv venv
> venv\Scripts\activate
```

✓ After activating, the line starts with (venv). I must see it every time I work!

- Why venv? All libraries of this project stay in one box and do not mix with other projects.

11 Install the light libraries

```
> pip install pandas numpy matplotlib scikit-learn tqdm
```

output: `Successfully installed ...`

```
> python -c "import pandas, numpy, sklearn; print('all good')"
```

output: `all good`

- pandas = tables/data. numpy = maths. matplotlib = graphs. scikit-learn = metrics. tqdm = progress bar.

12 Check for a GPU

```
> nvidia-smi
```

output: `'nvidia-smi' is not recognized ...`

- So my laptop has no NVIDIA GPU. That is OK!

✓ Plan: write + test code on laptop (CPU). Train the big runs on free Google Colab / Kaggle GPU.

6. PyTorch + Transformers

the AI libraries

13 Install PyTorch (CPU version)

```
> pip install torch --index-url https://download.pytorch.org/whl/cpu
```

- Big download (few hundred MB). Wait a few minutes.

```
> python -c "import torch; print(torch.__version__)"
```

output: `2.14.1+cpu`

WATCH OUT: My prompt showed NO (venv) at that moment. If (venv) is missing, the library goes to the wrong place. Fix: run venv\Scripts\activate, then install again.

14 Install transformers + sentencepiece

```
> pip install transformers sentencepiece
```

```
> python -c "import transformers; print(transformers.__version__)"
```

output: `5.20.1`

What each one does:

- torch - the deep-learning engine (models, training).
- transformers - Hugging Face library. Gives us T5-small and BERT-small.
- sentencepiece - the tokenizer T5 needs (cuts text into pieces).
- pymatgen - crystal library. NOT installed yet, we add it when needed.

TIP: My project is inside OneDrive. OneDrive tries to sync thousands of venv files and can slow the laptop. If slow: right-click the OneDrive icon near the clock > Pause syncing.

7. Save the setup to GitHub

.gitignore, requirements.txt, notes.md

15 Make .gitignore (files Git must skip)

```
> echo venv/> .gitignore
> echo data/>> .gitignore
> echo __pycache__/>> .gitignore
> echo *.pt>> .gitignore
```

- venv/ = thousands of library files. data/ = big datasets. __pycache__/ = junk. *.pt = heavy model files.

16 Save my library list

```
> pip freeze > requirements.txt
> git status
```

- requirements.txt = list of my libraries. Rebuild anywhere: pip install -r requirements.txt

✓ git status showed only .gitignore + requirements.txt. No venv = ignore works!

```
> git add .gitignore requirements.txt
> git commit -m "Day 1: add environment setup and gitignore"
> git push
```

17 Write my first note

```
> echo # Project Notes> notes.md
> echo.>> notes.md
> echo LLM-Prop takes a text description of a crystal as input ... >> notes.md
```

(the full sentence: text of a crystal in, properties like band gap and volume out)

18 Commit + push the note

```
> git add notes.md
> git commit -m "Day 1: add project notes"
> git push
```

✓ Refreshed GitHub: notes.md is there. Day 1 DONE!

8. Quick cheat sheet

commands I will use every day

`git status`	= what changed? (red = not saved yet)
`git add .`	= put all changed files in the basket
`git commit -m "msg"`	= save a snapshot with a message
`git push`	= upload my commits to GitHub
`git remote -v`	= show the GitHub link
`venv\Scripts\activate`	= turn ON my venv (see (venv))
`pip install <name>`	= install a library
`pip freeze > requirements.txt`	= update my library list

Common problems

and the fix

- x (venv) not showing in the prompt
  - fix: run venv\Scripts\activate
- x 'python' / 'git' not recognized
  - fix: install it, then reopen the terminal
- x 'Please tell me who you are'
  - fix: git config --global user.name / user.email
- x git push asks for login
  - fix: sign in to GitHub in the pop-up window
- x Laptop slow while installing
  - fix: pause OneDrive syncing for a few hours
- x venv appears in git status
  - fix: STOP, check .gitignore has venv/

9. End of Day 1

where I stand now

My project folder now:

```
LLM_PROP/
    README.md           (on GitHub)
    .gitignore          (on GitHub)
    requirements.txt    (on GitHub)
    notes.md            (on GitHub)
    venv/               (only on my laptop, ignored)
```

My 3 commits today:

✓ Day 1: add README

✓ Day 1: add environment setup and gitignore

✓ Day 1: add project notes

Versions installed:

- Python 3.11.9 | PyTorch 2.14.1 (CPU) | transformers 5.20.1

Next: Day 2 (data!)

- Open the arXiv page 2310.14029 > click 'this https URL' under the abstract (the authors' code page).
- Find the TextEdge dataset there (look for a data / dataset folder or link).
- Download it, put it in LLM_PROP/data/ (git ignores it), and look at the columns.

20-day map:

- Days 2-3: data | 4-6: model + training code | 7-10: train 3 tasks
- Days 11-13: experiments | 14-16: tuning + compare | 17-20: demo, report, viva

Great start! Keep the daily GitHub streak going. ✳ ✳